Semantic Fragmentation and Constitutional Structure in Software Systems

Diagnosing Meaning Loss Across Distributed Constitutional Systems

SFA does not primarily grade correctness or performance.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Modern software systems exhibit a persistent gap between their apparent structure and their actual governing behavior. Modules, APIs, configuration schemas, and documentation describe visible surfaces, while configuration resolution, authority delegation, execution routing, fallback behavior, and exception handling often remain distributed through implementation. The system has an operative constitution, but may be unable to represent or explain it.

This paper introduces **Semantic Fragmentation Analysis (SFA)** as a source-bounded method for characterizing how a software system distributes, obscures, contradicts, or externalizes its governing structure. SFA does not primarily grade correctness or performance. It identifies decision classes, authority surfaces, admissibility rules, resolution cascades, fallbacks, projections, evidence, and revision paths; reconstructs representative decision events; and reports where declared structure and operative behavior diverge.

The paper defines semantic fragmentation, operative and declared constitutions, structural legibility, resolution surfaces, authority scopes, and constitutional exposure. It formalizes a decision event, gives a non-compensatory fragmentation profile, states diagnostic invariants, expands the pattern taxonomy, and supplies a worked configuration-resolution example. It then locates SFA within the full Possibility–Specification–CM1–CM2–Instance Configuration–Implementation Realization–Evidence–Revision funnel and connects local structural legibility to constitutional compatibility across systems.

SFA is a diagnostic architecture, not a certification result. Explicit models can be stale, incomplete, deceptive, excessively centralized, privacy-invasive, or more costly than the opacity they replace. The method therefore preserves uncertainty, hidden remainder, counterevidence, and refusal. Its intended contribution is a disciplined route from implicit constitutional behavior toward software whose

authority, resolution, transformation, and failure semantics can be inspected and revised without claiming that legibility alone makes the software correct, secure, legitimate, or desirable.

Keywords: semantic fragmentation; constitutional computing; software architecture; authority; admissibility; resolution surface; fallback; structural legibility; provenance; interoperability.

1. Introduction: The Problem of Implicit Structure

Software systems are typically described through functions, classes, APIs, interfaces, configuration schemas, and documentation. These surfaces create the impression that a system's structure is explicit and well defined. Yet actual behavior also depends on deeper decision processes:

- how configuration values are resolved;
- which source or component has authority;
- how execution paths are selected;
- which exceptions trigger fallback;
- how defaults, environment, state, and history interact;
- what evidence remains after a decision takes effect.

These processes often exist as distributed logic embedded in implementation. A system can therefore appear modular while its real control structure crosses modules; appear configurable while hidden defaults dominate; or appear governed by one policy while fallback paths implement another.

The resulting mismatch is not adequately described by code complexity alone. Complexity concerns how much structure must be understood. **Semantic fragmentation** concerns whether the governing meaning of that structure–who may decide what, under which conditions, by which rule, with what fallback and evidence–exists as a coherent inspectable object.

A fragmented system is not necessarily incorrect. It may perform well, pass tests, and satisfy users. The narrower claim is that it lacks **structural legibility**: an accountable ability to answer how a consequential outcome was selected. That distinction matters because a system can be functionally successful yet difficult to audit, repair, translate, replace, or govern.

The central thesis is:

> Software systems commonly possess an operative constitutional form whose authority, admissibility, resolution, routing, and fallback rules are distributed across implementation. Semantic Fragmentation Analysis reconstructs and tests that form without presuming that a successful reconstruction is complete or normatively justified.

2. Definitions and Claim Boundary

2.1 Semantic fragmentation

Semantic fragmentation is the condition in which governing rules of decision, authority, admissibility, precedence, transformation, fallback, or evidence are distributed across implicit, inconsistent, opaque, or externally represented forms rather than available through a coherent declared model.

Fragmentation may be locational, when rules are scattered; semantic, when the same term or status has incompatible meanings; constitutional, when authority or admissibility is undeclared; evidentiary, when outcomes cannot be reconstructed; or projective, when the interface presents a simpler constitution than the implementation exercises.

Fragmentation is relative to a bounded question and observation surface. No finite analysis proves that every operative rule has been recovered. SFA must therefore report both characterized structure and **remainder**: behavior, authority, state, dependencies, and environmental influence not yet explained.

2.2 Operative and declared constitutions

The **operative constitution** of a system is the set of rules and relations that actually govern consequential transitions in the observed scope. It includes code, configuration precedence, defaults, runtime state, external services, human intervention, deployment environment, fallback paths, and any other factor that can alter an outcome.

The **declared constitution** is the system's explicit account of those rules: schemas, policy artifacts, interface contracts, intermediate representations, help text, documentation, diagnostic surfaces, and machine-readable declarations.

The two need not be identical in form, but consequential differences must be identifiable. A declared model that omits a harmless implementation optimization may remain adequate. A model that says a policy controls an outcome while an undocumented fallback bypasses it is constitutionally misleading.

2.3 Structural legibility

Structural legibility is the bounded capacity to reconstruct a consequential outcome from declared candidates, authority bases, selection rules, fallbacks, relevant state, and evidence. Legibility is not the same as simplicity, transparency of every secret, or unrestricted disclosure. A secure system may legitimately hide sensitive values while still exposing typed reasons, authority, scope, and verifiable evidence.

2.4 Resolution surface

A **resolution surface** is an explicit human- or machine-legible interface through which a system reports what was resolved, which candidates were considered,

what authority and precedence rules applied, which fallback or refusal occurred, what outcome was selected, and what evidence supports the report.

A log line is not automatically a resolution surface. The surface must preserve the semantics required to reconstruct or challenge the decision. Conversely, a resolution surface need not reveal protected values; it may use commitments, redaction classes, or authorized views.

2.5 Claim boundary

SFA produces a structural characterization under declared scope, observation, and test conditions. It does not by itself establish:

- functional correctness;
- security or absence of hidden paths;
- normative legitimacy of the governing rules;
- completeness of the reconstructed constitution;
- identity between a model and the system;
- constitutional compatibility with another system;
- improvement from a proposed refactor.

Those are separate claims requiring separate evidence.

3. A Minimal Operational Model

Let a consequential resolution event be represented as:

$$r = \langle q, C, A, P, F, x, o, E \rangle, \tag{1}$$

where:

- q is the question or decision class;
- C is the set of considered candidates;
- A records the authority basis and scope for each candidate or rule;
- P is the precedence or selection procedure;
- F is the declared fallback or refusal behavior;
- x is the relevant input, environment, and prior state;
- o is the selected outcome or refusal;
- E is the retained evidence sufficient for the bounded claim.

The operative constitution observed across a scoped set of events R can be approximated by the authority, precedence, fallback, and transition relations recovered from those events:

$$K_{\mathrm{op}}(S, R) = \bigcup_{r \in R} \{A_r, P_r, F_r, T_r\}. \tag{2}$$

The declared constitution is $K_{\mathrm{decl}}(S)$. Two diagnostic differences are then useful:

$$\Delta_{\text{hidden}} = K_{\text{op}} \setminus K_{\text{decl}}, \qquad \Delta_{\text{stale}} = K_{\text{decl}} \setminus K_{\text{op}}. \tag{3}$$

Δ_{hidden} identifies recovered operative rules absent from the declaration. Δ_{stale} identifies declared rules not observed in the scoped operative analysis. Neither set is automatically a defect: observation may be incomplete, a declared path may be rare, or an implementation detail may be semantically irrelevant. Each member requires disposition and evidence.

SFA should report a profile rather than one compensatory score:

$$\Phi(S) = \langle L, D, A_o, T_d, P_d, R_d \rangle, \tag{4}$$

where L is locational dispersion, D decision opacity, A_o authority opacity, T_d trace deficit, P_d projection divergence, and R_d revision deficit. A low value on one axis must not cancel a critical failure on another. No validated universal measurement scale is claimed here; Section 11 gives candidate operationalizations.

4. Diagnostic Invariants

The following are proposed SFA invariants for an explicitly governed resolution architecture. They are evaluation obligations, not claims that current software satisfies them.

1. **Authority is declared.** Every consequential selection identifies the authority basis and its scope, or explicitly reports that authority is unresolved.
2. **Candidates are bounded.** The considered candidate set and any intentionally omitted class are recoverable at the resolution required by the claim.
3. **Precedence is explicit.** Selection order, tie handling, and override semantics are represented separately from candidate values.
4. **Fallback is constitutional.** Defaults, error recovery, compatibility paths, and emergency overrides are treated as governing rules rather than invisible implementation details.
5. **Effects are attributable.** A consequential effect can be linked to the resolution event that admitted it without implying that the event proved the decision correct.
6. **Projection divergence is detectable.** A user-facing or API-facing account cannot silently substitute for materially different operative behavior.
7. **Scope boundaries are explicit.** Distributed authorities identify where their jurisdiction begins, ends, overlaps, or requires escalation.
8. **Refusal and remainder persist.** Missing evidence, unresolved authority, refused action, and unexplained behavior are not rewritten as success or zero.
9. **Revision preserves lineage.** A changed rule, fallback, schema, or authority relation creates a successor state with its predecessor and migration consequences retained.

A system may intentionally relax an invariant. SFA then records the relaxation, rationale, scope, and consequence rather than silently treating the invariant as met.

5. Semantic Fragmentation Analysis

An SFA pass is bounded by a target, decision classes, runtime conditions, and evidence ceiling. It proceeds as follows.

5.1 Scope the analysis

Declare the system head, build, configuration, environment, external services, user role, relevant secrets or redactions, and the exact decision classes under study. A claim about configuration resolution must not become a claim about every authority path in the system.

5.2 Inventory visible and operative surfaces

Record the modules, APIs, schemas, documentation, policy artifacts, configuration sources, runtime stores, logs, deployment controls, human procedures, and external providers that may influence the decision. Mark which are declarations, projections, evidence, or effect-producing mechanisms.

5.3 Enumerate decision classes

Examples include configuration value selection, resource choice, identity resolution, route selection, parser choice, capability admission, policy evaluation, fallback, retry, override, and termination. Each class receives an expected decision-event schema based on equation (1).

5.4 Trace representative resolutions

For each class, choose ordinary, boundary, conflicting, missing, denied, stale, and failure cases. Trace candidates, authority, precedence, fallback, state, outcome, and retained evidence. Where a field cannot be reconstructed, preserve `UNKNOWN`, `WITHHELD`, or `NOT_OBSERVED` rather than inventing a value.

5.5 Map authority and scope

Identify which component or actor may propose, select, override, execute, observe, interpret, and revise. A module that computes a value, a policy that authorizes it, and a runtime that applies it may be different authorities.

5.6 Compare projection with operation

Compare documentation, UI explanation, API response, or declared policy with the traced operative path. Differences receive typed dispositions: benign

abstraction, protected omission, stale declaration, undocumented authority, unresolved remainder, or material contradiction.

5.7 Probe counterfactuals

Change one candidate, authority, precedence rule, fallback condition, or environment fact at a time where safely possible. Verify whether the observed outcome changes as the reconstructed model predicts. Counterfactual mismatch is evidence that the model is incomplete, not permission to patch the explanation after the fact.

5.8 Produce a characterization

The output includes scoped decision classes, recovered resolution events, authority map, fragmentation profile, hidden and stale differences, pattern findings, evidence, remainder, failure cases, and proposed successor changes. It is a diagnostic package, not a pass/fail certificate.

6. Structural Pattern Taxonomy

The source identifies three initial patterns. The expanded taxonomy keeps those and proposes additional comparison classes.

6.1 Implementation-derived authority

Authority is not declared but emerges from function boundaries, control flow, mutable state, call order, or deployment privilege. Code becomes the only source of truth for governance, even when documentation implies a separate policy source.

6.2 Structural contrast amplification

One domain is strongly formalized while an adjacent domain remains implicit. The explicit subsystem makes the neighboring informality visible: for example, a typed policy model may reveal that fallback and operator override remain conventional and unreceipted.

6.3 Resolution cascade opacity

Each precedence cascade may be locally coherent, but several interacting cascades–configuration, mode, resource, identity, plugin, or environment resolution–lack a unified diagnostic surface. Users can observe the outcome without reconstructing why it occurred.

6.4 Fallback constitutionalization

A fallback designed as exceptional becomes the ordinary governing path. Because it is labeled fallback, its authority, testing, or observability may remain weaker

than the primary path even while it determines most effects.

6.5 Shadow constitution

Documentation, schemas, or policy artifacts describe one rule set while implementation, deployment, or operator practice consistently exercises another. The declared constitution becomes a projection rather than the operative source of governance.

6.6 Scope leakage

Authority valid in one semantic space influences another without an explicit bridge. A renderer changes domain state, a parser determines authorization, or a deployment default overrides a user-level policy outside its declared jurisdiction.

6.7 Authority aliasing

Several components appear to exercise the same named authority but derive it from different warrants or scopes. The shared label hides competing constitutional bases.

6.8 Evidence truncation

The system records the selected outcome but discards rejected candidates, precedence, fallback, relevant state, or refusal. Post hoc explanation then becomes narrative reconstruction rather than evidence-backed resolution.

6.9 Projection laundering

A legible UI, API response, or generated explanation is treated as proof that the underlying decision was itself explicit and governed. Presentation quality substitutes for operative evidence.

6.10 Revision discontinuity

Changes to precedence, authority, schema, or fallback overwrite prior behavior without a successor relation, migration account, or compatibility statement. The system cannot explain whether old and new outcomes belong to one constitutional lineage.

These patterns may co-occur. They are diagnostic candidates whose presence must be shown by bounded evidence rather than assigned by resemblance alone.

7. Authority and Spatial Structure

A useful model is hub-and-spoke coordination with spatially scoped authority. A hub may orchestrate execution while spokes govern configuration, parsing,

identity, policy, execution, evidence, or rendering. The model is useful only if it does not collapse orchestration into sovereignty.

For each semantic space, SFA asks:

- what objects and effects lie within the scope;
- who may propose, decide, override, and revise;
- which invariants apply;
- how requests cross the boundary;
- what evidence and refusal return;
- how overlap or conflict is resolved.

Fragmentation arises when scopes are unclear, authority is implicit, cross-space translation is lossy, or coordination is visible only as a final output. Centralizing every decision is not the necessary remedy. A distributed constitution can remain legible if its scopes, bridges, and escalation rules are explicit.

8. Resolution as a First-Class Concept

Software constantly resolves which value, identity, resource, route, policy, or implementation applies. Treating resolution as an implementation detail hides the point at which competing semantic possibilities become one operative effect.

A resolution surface should expose at least:

```
decision class
scope and exact instance
candidate identities and sources
authority basis for each candidate
precedence or selection rule
rejected and unavailable candidates
fallback or refusal path
selected outcome
effect status
evidence and uncertainty
revision or appeal route
```

The surface should distinguish **proposal**, **admission**, **execution**, **observation**, and **interpretation**. A selected value may never be applied. A runtime may apply a value that the diagnostic surface failed to observe. A human explanation may misinterpret an accurate trace. Combining these stages produces false confidence.

9. Worked Example: Interacting Configuration and Resource Resolution

Consider a stipulated media application with a `mode` setting and an independently selected playback engine. The visible documentation says command-line configu-

ration has highest precedence, followed by project configuration, environment, user configuration, and default.

For one run:

- **Input:** command line
  Candidate: `mode=safe`
  Authority basis: current invocation
  Availability: present
- **Input:** project file
  Candidate: none
  Authority basis: project scope
  Availability: absent
- **Input:** environment
  Candidate: `mode=fast`
  Authority basis: process environment
  Availability: present
- **Input:** user file
  Candidate: `mode=balanced`
  Authority basis: user scope
  Availability: present
- **Input:** default
  Candidate: `mode=balanced`
  Authority basis: built-in fallback
  Availability: present

The mode resolver selects `safe`. A second cascade then chooses a playback engine. The preferred safe engine is unavailable, so a compatibility fallback selects the legacy engine. The UI reports only

`mode=safe`.

The result is not necessarily wrong. It is fragmented if the system cannot answer that two independent resolutions occurred, that the engine fallback had separate authority, or that `safe` at the mode layer did not entail the preferred safe engine at the resource layer.

A bounded resolution record would preserve:

```
MODE-RESOLUTION
  selected: safe
  source: command line
  rejected: environment=fast, user=balanced, default=balanced
  rule: cli > project > environment > user > default

ENGINE-RESOLUTION
  requested profile: safe
```

```
preferred engine: unavailable
selected: legacy-engine
basis: compatibility fallback F-3
effect: playback initialized
warning: safe-mode projection does not imply preferred-engine
realization
```

An SFA counterfactual removes the command-line candidate and predicts environment selection, then makes the preferred engine available and predicts withdrawal of fallback F-3. If either prediction fails, the reconstructed constitution is incomplete. This example illustrates the method; it is not evidence about a real application.

10. Constitutional Software Architecture and the Realization Funnel

SFA diagnoses implicit constitutional structure. Constitutional software architecture asks how selected governing relations can become explicit, inspectable, and revisable without confusing representation with reality.

Stage	Semantic-fragmentation responsibility
Possibility	Explore possible authority, resolution, fallback, observability, and compatibility structures, including opaque, centralized, conflicting, harmful, and minimal forms.
Specification	Define decision classes, scopes, invariants, event schemas, authority grammar, evidence kinds, refusal, remainder, and claim ceilings.
CM1	Enumerate all admissible configurations that satisfy the specification, including undesirable distributions, weak exposure, excessive disclosure, and high-cost instrumentation.
CM2	Prefer configurations that minimize material hidden authority, preserve privacy and security, expose consequential fallback, support challenge and revision, and avoid unnecessary centralization.

Stage	Semantic-fragmentation responsibility
Instance Configuration	Select exact sources, precedence, scopes, bridges, redactions, evidence retention, runtime targets, observers, and test cases for one system instance.
Implementation Realization	Implement the policies, HLIR/MCIR forms, resolvers, kernels, traces, interfaces, and migration paths chosen for that instance.
Evidence	Execute ordinary, conflict, missing, denied, fallback, stale, adversarial, and counterfactual cases; compare declared and observed events; preserve unexplained remainder.
Revision	Create successor specifications and configurations when rules or findings change; retain predecessor behavior, migration decisions, incompatibilities, and unresolved findings.

The funnel prevents several category errors. A specification is not a running resolver. A running resolver is not evidence that every effect passed through it. A complete trace is not proof that the policy was legitimate. A preferred CM2 architecture is not the only technically admissible CM1 realization.

Human-legible intermediate representations, machine-legible constitutional forms, governed execution kernels, and resolution surfaces can reduce fragmentation only when their relation to operative effects is verified. Otherwise they become another declared layer over a shadow constitution.

11. From Local Legibility to Constitutional Compatibility

Interoperability depends on more than shared message formats. The companion paper *Constitutional Compatibility: A Meta-Map for Institutional Semantic Standards* analyzes systems across semantic boundary, authority grammar, identity, admissibility, translation law, receipt structure, and constitutional exposure [CC].

SFA supplies a local diagnostic prerequisite: before comparing two constitutions, each system must expose enough of its operative structure to identify what is being compared. This does not require full global compatibility. Compatibility is scoped to an operation, artifact, or relationship.

For systems S_1 and S_2, a proposed bridge M is admissible for operation u only if the translation preserves the declared invariants required by each side for u, or explicitly refuses the operation:

$$\begin{aligned} &\mathrm{Compat}_u(S_1, S_2, M) \\ &\Rightarrow \mathrm{Preserves}_M(I_1^u) \quad . \\ &\wedge \mathrm{Preserves}_M(I_2^u) \end{aligned} \tag{5}$$

This implication is an obligation schema, not an executable test or proof of compatibility. SFA can reveal missing authority, scope conflict, identity ambiguity, or unreceipted translation that prevents the obligation from being evaluated. A compatibility bridge then becomes an explicit constitutional artifact rather than an undocumented adapter.

12. Evidence and Evaluation

Candidate observations for the fragmentation profile include:

- **Axis:** Locational dispersion
  Candidate observations: distinct files, stores, services, procedures, and actors required to recover one decision
  Nonclaim: more locations do not automatically mean worse architecture
- **Axis:** Decision opacity
  Candidate observations: proportion of scoped events lacking candidates, rule, fallback, or outcome linkage
  Nonclaim: a complete-looking trace may still be false
- **Axis:** Authority opacity
  Candidate observations: events without a resolvable authority basis and scope
  Nonclaim: declared authority is not necessarily legitimate
- **Axis:** Trace deficit
  Candidate observations: inability to connect effect to admitted decision event
  Nonclaim: traceability does not prove correctness
- **Axis:** Projection divergence
  Candidate observations: material differences between declared/presented and observed behavior
  Nonclaim: abstraction is not automatically deception
- **Axis:** Revision deficit
  Candidate observations: changes without predecessor, migration, or compatibility relation
  Nonclaim: versioning alone does not preserve meaning

Evaluation should bind the exact source head, build, configuration, environment, permissions, external dependencies, and observation tools. Tests should include ordinary operation, conflicting candidates, absent values, invalid inputs, authority

denial, fallback activation, stale cache or state, partial failure, external-service change, and counterfactual perturbation.

Results should report coverage and remainder. A system that explains one carefully selected happy path has not established general legibility. A system that refuses every uncertain case may avoid false explanations while remaining unusable. Coverage, error within covered cases, refusal quality, overhead, privacy loss, and repair value must be evaluated separately.

Hidden-Constitution Diagnostic Theory and Method

Across the completed P1 corpus, Semantic Fragmentation and Constitutional Structure is the **diagnostic theory and method for reconstructing hidden operative constitutions from dispersed authority, precedence, fallback, resolution, effect, evidence, projection, and revision surfaces in software systems**. It defines the method family; it is distinct from Semantic Field Analysis in general and from the completed seventeen-system evidence study.

- **Adjacent work:** *Semantic Field Analysis*
  Supplies: A general semantic mapping procedure across entities, relations, authority, evidence, and hidden structure.
  SFA boundary: Software SFA specializes that method to consequential resolution and operative constitutions.
- **Adjacent work:** *What Software Systems Actually Receipt, Govern, and Hide*
  Supplies: A frozen seventeen-system comparative evidence study.
  SFA boundary: The study tests and revises SFA categories without becoming the method's identity.
- **Adjacent work:** *Constitutional Binding Theory: Governed Anchor Resolution as a Universal Pattern in Semantic Systems*
  Supplies: Detailed analysis of one resolution family: governed anchor binding.
  SFA boundary: SFA generalizes across configuration, authority, routing, fallback, resources, and effects.
- **Adjacent work:** *The Constitutional Observability Platform*
  Supplies: Heterogeneous evidence surfaces and typed correspondence edges.
  SFA boundary: COP can expose operative state; SFA diagnoses whether the resulting account is structurally complete enough.
- **Adjacent work:** *Governance Coverage Gaps*
  Supplies: Capability and effect paths outside declared governance.
  SFA boundary: GCG tests path coverage; SFA reconstructs the broader operative constitution and its fragmentation.

An SFA investigation should bind:

$$S = (system_head, scope, decision_classes, surfaces, \\ resolution_events, authority, precedence, fallback, \\ effects, evidence, projections, counterfactuals, \\ remainder, profile, revision)$$

The core comparison is among declared, operative, projected, and reconstructed constitutions. These can diverge without fraud: documentation may be stale, observation incomplete, modularity intentional, or abstraction legitimate. The method should classify each divergence and retain uncertainty rather than rewarding centralization or maximum disclosure.

Fragmentation is material when it obstructs correct resolution, challenge, repair, migration, compatibility, user sovereignty, or accountability. Location count alone is not a defect. The noncompensatory profile should preserve locational dispersion, decision opacity, authority opacity, trace deficit, projection divergence, and revision deficit rather than collapse them into one score.

The universal GSM can type
`DECLARES_SOFTWARE_CONSTITUTION`,
`OPERATES_UNDER_HIDDEN_RULE`, `RECONSTRUCTS_RESOLUTION_EVENT`,
`DERIVES_AUTHORITY_FROM_IMPLEMENTATION`, `ACTIVATES_CONSTITUTIONAL_FALLBACK`,
`ALIASES_AUTHORITY`, `TRUNCATES_EFFECT_EVIDENCE`, `LAUNDERS_PROJECTION`,
`PRESERVES_UNEXPLAINED_REMAINDER`,
and
`REVISES_SFA_CHARACTERIZATION`. These relations support both diagnosis and later constitutional refactoring.

This integration establishes four editorial commitments: retain the paper as the hidden-constitution diagnostic theory and software method; adopt the investigation envelope and four-constitution comparison; define material fragmentation by impaired consequence rather than dispersion alone; and keep general Semantic Field Analysis, software SFA theory, the seventeen-system study, and any tooling as distinct linked identities.

Custodied Reconstruction Under Agentic and Adversarial Change

The fixed-seed comparison with Work Custody, ACG, Constitutional Epistemology under adversarial pressure, and Proxy Collapse makes a fragmentation investigation both responsibility-bearing and calibration-aware:

$$F = (subject, scope, declared_constitution, \\ operative_constitution, projection, reconstruction, \\ custodian, threats, proxy_assumptions, evidence, \\ remainder, repair, revision)$$

Work Custody identifies who must advance the reconstruction, obtain missing evidence, challenge an inference, repair a surface, or return unresolved remainder. Custody over a report is not authority over the analyzed system and does not make the reconstruction complete.

ACG can use the fragmentation map to select assurance subjects, courts, effect probes, and revision gates. Constitutional Epistemology adds adversarial cases in which a formally consistent declaration or receipt intentionally or inadvertently hides the operative path through omission, drift, witness capture, or staged observation.

A declared or projected constitution is often used as a proxy for operative behavior. When generators, deployment paths, incentives, or environments change, that proxy can lose calibration even if its syntax and lineage remain intact. Proxy Collapse supplies this bounded diagnostic; it does not imply that all documentation or models are invalid.

The universal GSM can type
`CUSTODIES_CONSTITUTION_RECONSTRUCTION`,
`INFORMS_ACG_ASSURANCE_SCOPE`, `ATTACKED_BY_SEMANTIC_PRESSURE`, `USES_PROJECTION_AS_PROXY`, `LOSES_OPERATIVE_CALIBRATION`, `RETAINS_UNEXPLAINED_REMAINDER`,
and
`REPAIRS_FRAGMENTED_SURFACE`.

This randomized review establishes four editorial commitments: assign custody to reconstruction and repair work; feed the operative map into ACG scope; adversarially test declared/operative divergence; and treat projected constitutions as recalibratable proxies rather than guaranteed correspondence.

A Custodied Reconstruction Profile for Hidden Constitutions

Twelve retained downstream uses place software semantic-fragmentation analysis before custody repair, adversarial epistemology, ACG court selection, GCML and GCNL design, legacy migration, proxy recalibration, binding-quality review, semantic-debt accounting, general field analysis, semantic infrastructure, and the comparative system study. They confirm that the paper defines a diagnostic reconstruction profile, not a mandate to centralize every rule or disclose every protected fact.

$$
\begin{aligned}
D^* = (&subject_head, scope, decision_classes, surfaces,\\
&declared, operative, projected, reconstructed, authority,\\
&precedence, fallback, effects, evidence, counterfactuals,.\\
&custody, threats, protected_omissions, remainder,\\
&repair, revision)
\end{aligned}
$$

Fragmentation is material only relative to a consequence: impaired resolution, challenge, repair, migration, compatibility, sovereignty, accountability, or evidence. Distribution can be legitimate modularity. A unified model can itself become a shadow constitution, suppress local authority, expose secrets, or drift from operation. The profile therefore reports noncompensatory dimensions and coverage rather than one universal fragmentation score.

Reconstruction is a claim-bearing activity. Each finding binds an exact system generation, environment, observer, decision class, evidence path, counterfactual, confidence, and custodian. `UNKNOWN`, protected non-disclosure, external influence, and unresolved remainder are valid outcomes. A repair proposal remains separate from authorization to change the system and from evidence that operative behavior actually changed.

The universal GSM can type
`INSTANTIATES_FRAGMENTATION_REVIEW`,
`RECONSTRUCTS_OPERATIVE_CONSTITUTION`, `DISTINGUISHES_LEGITIMATE_DISTRIBUTION`,
`PRESERVES_PROTECTED_OMISSION`, `ASSIGNS_RECONSTRUCTION_CUSTODY`,
`TESTS_WITH_COUNTERFACTUAL`, `PROPOSES_FRAGMENTATION_REPAIR`,
and
`WITHHOLDS_COMPLETE_CONSTITUTION_CLAIM`.

This review establishes four editorial commitments: adopt the complete reconstruction record; define materiality by scoped consequence rather than dispersion; require custody, threat, counterfactual, coverage, and protected-omission fields; and keep diagnosis, repair proposal, authorized change, and observed effect separate.

13. Failure Modes, Counterevidence, and Limits

SFA and explicit constitutional architecture can fail in several ways:

- **model staleness:** the declared structure stops tracking implementation;
- **instrumentation theater:** rich traces create confidence without binding effects;
- **centralization:** one canonical model suppresses legitimate local authority or plurality;
- **privacy or security leakage:** traces reveal protected values, topology, or attack surfaces;
- **observability effect:** instrumentation changes timing or decisions being measured;
- **cost displacement:** maintenance and runtime overhead exceed the value of legibility;
- **explanatory compression loss:** a compact account omits details material to a dispute;
- **false closure:** `UNKNOWN` or external influence is silently assigned to the nearest component;

- **framework duplication:** SFA adds vocabulary without outperforming established architecture, provenance, policy, tracing, or debugging methods;
- **outcome irrelevance:** greater legibility does not improve repair time, policy compliance, compatibility, user sovereignty, or error detection.

Evidence against the stronger program would include repeated findings that ordinary architecture documentation and tracing recover the same decisions at lower cost; inability to distinguish material fragmentation from normal modularity; low inter-reviewer agreement on pattern classification; resolution surfaces that drift as rapidly as prose documentation; failure of counterfactual probes to validate reconstructed models; or no measurable benefit in debugging, migration, audit, or cross-system integration.

The present paper performs no external literature review and reports no empirical SFA study. For a future evidence-qualified revision, it requires comparison with current work in software architecture, configuration management, provenance, policy-as-code, access control, observability, program analysis, event sourcing, distributed tracing, safety engineering, interoperability, and human-computer explanation.

Reconstruct Hidden Constitutions Without Creating a New One

Public orientation

Semantic Fragmentation Analysis reconstructs how software actually resolves authority, configuration, routing, fallback, transformation, and evidence when the governing behavior is scattered across code and environment. It can expose gaps between the declared and operative constitution. The reconstruction is itself a bounded claim and can become another stale, centralized, or deceptive layer.

Adversarial tests

1. **Normal-modularity attack.** Legitimate distribution of responsibility is scored as fragmentation merely because rules are not centralized.
2. **Shadow-constitution attack.** The SFA model becomes the authoritative story while operative behavior, local authority, and exceptions continue elsewhere.
3. **Stale-declaration attack.** The reconstruction was accurate for one generation but remains active after code, configuration, provider, or environment change.
4. **Instrumentation-theater attack.** Rich traces and diagrams appear explanatory without binding the observed effect or testing counterfactuals.
5. **Protected-omission attack.** Legibility is pursued by exposing secrets, personal data, security topology, or legitimately restricted decisions.

6. **False-closure attack.** Unknown, inaccessible, nondeterministic, human, or external influence is assigned to the nearest known component.
7. **Repair-authority attack.** A diagnostic finding is treated as authorization to centralize, rewrite, or deploy a repair.
8. **Framework-duplication attack.** Existing architecture, tracing, provenance, policy, configuration, and debugging methods recover the same decisions at lower cost.

Claim and evidence ceiling

SFA is a source-bounded diagnostic reconstruction profile. It does not certify completeness, correctness, security, legitimacy, compatibility, or desirability. Structural legibility is consequence-scoped and noncompensatory; protected omission and `UNKNOWN` are valid outcomes. Diagnosis, repair proposal, authorized change, and observed effect remain separate.

Claims about prevalence, reviewer reliability, debugging benefit, migration improvement, policy compliance, user comprehension, sovereignty, or comparative novelty require external literature and empirical studies over exact system generations. No external research was added in this pass.

Reader-facing consequence

The public value is the ability to ask why a consequential software decision happened and receive a custodied, testable reconstruction - including what remains hidden or unknown - without pretending that explanation itself makes the system legitimate.

14. Research Agenda

1. **Pattern taxonomy.** Test whether the proposed patterns are distinguishable, reproducible, and useful across systems and reviewers.
2. **Resolution modeling.** Develop minimal decision-event schemas for configuration, identity, routing, policy, fallback, and resource selection without forcing one universal implementation.
3. **Authority representation.** Separate proposal, authorization, execution, observation, interpretation, override, and revision authority across scoped semantic spaces.
4. **Constitutional exposure.** Determine which structure must be public, participant-visible, operator-visible, auditor-visible, machine-verifiable, committed but redacted, or legitimately undisclosed.
5. **Metrics.** Validate non-compensatory profile axes, coverage measures, reviewer agreement, and outcome correlations rather than inventing an unsupported universal score.
6. **Cross-system analysis.** Apply SFA before constitutional-compatibility mapping and measure whether explicit bridges reduce translation and authority failures.

7. **Refactoring experiments.** Compare unchanged systems, documentation-only changes, resolution surfaces, explicit constitutional artifacts, and governed kernels on repair, migration, audit, and user-comprehension tasks.
8. **Adversarial analysis.** Test shadow paths, stale declarations, forged traces, fallback dominance, authority aliasing, incomplete observers, and protected information.

15. Corpus Integration

This paper occupies a diagnostic bridge among several corpus layers without merging their identities:

- **Constitutional Computing** supplies the larger proposal that software effects should be organized through explicit authority, admissibility, evidence, refusal, and revision rather than hidden implementation convention [CComp].
- **Sovereign Software** turns structural legibility toward user inspection, substitution, renegotiation, provider exit, and the hardware boundary; SFA can identify where those capabilities are blocked by hidden constitutional behavior [SS].
- **GMB**, the current Governed Meaning Base in the historical `CCF -> CMB -> GMB` lineage, can hold standing-aware concepts, rules, conflicts, evidence, and revision history. A GMB record is not proof that operative code follows it [GMB].
- **GSOL and MCIR** can provide specification and intermediate representations for authority- and effect-bearing operations. SFA tests whether their obligations survive lowering into configuration and implementation [GSOL].
- **Semantic Legitimacy Before Context** prevents observed or documented material from becoming binding merely because it is available to the analysis [SLC].
- **Reasoning by Way of Receipts** preserves intermediate decisions, refusals, evidence, and debt; receipts make resolution inspectable without proving truth or correctness [RWR].
- **The Three Hash Rule** separates exact state, admissibility commitments, and meaning commitments so byte identity cannot silently stand in for constitutional or semantic equivalence [THR].
- **The Constitutional Observability Platform** separates domain state, presentation state, accessibility, pixels or outputs, interaction, and interpretation; it supplies heterogeneous evidence surfaces for checking an SFA reconstruction [COP].
- **Constitutional Binding Theory** analyzes governed anchor resolution; SFA generalizes the diagnostic question across configuration, authority, routing, fallback, and other decision classes without treating every resolver as the same system [CBT].

- **Constitutional Compatibility** begins once local constitutions are legible enough to compare across semantic boundary, authority, identity, admissibility, translation, receipts, and exposure [CC].

The paper therefore reaches Specification, defines CM1 and CM2 selection concerns, and supplies one synthetic Instance Configuration. It does not establish an implemented SFA tool, a validated metric, a complete system constitution, or a certified compatibility bridge.

16. Conclusion

Software systems are governed by structures that are rarely made fully explicit. Their constitutions are not absent: they are distributed through code paths, configuration layers, defaults, runtime state, fallbacks, external services, deployment controls, and human practice.

Semantic Fragmentation Analysis makes that distribution a direct object of inquiry. It reconstructs bounded decision events, maps authority and scope, compares declaration with operation, probes counterfactuals, preserves remainder, and characterizes patterns without reducing the result to correctness or one score.

The architectural response is not simply to centralize every rule or generate more documentation. It is to make consequential resolution, authority, transformation, fallback, evidence, refusal, and revision explicit at the level required for inspection and challenge–and then verify that the declaration remains connected to operative effects.

That shift supports a form of constitutional software architecture in which systems can explain how they govern themselves while remaining honest about what the explanation omits. Legibility does not guarantee legitimacy. It makes legitimacy, compatibility, repair, and user sovereignty more tractable questions because the governing structure is no longer required to remain hidden inside implementation.

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Cm1.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Evidence.** This stage is outside the present paper's asserted contribution and evidentiary scope.

- **Possibility Space.** The present paper does not claim a complete possibility space treatment. That stage remains successor work and cannot be inferred from the architecture described here.
- **Revision.** The present paper does not claim a complete revision treatment. That stage remains successor work and cannot be inferred from the architecture described here.

Corpus-Wide Reference Positioning

This paper argues that modern software systems exhibit a persistent gap between their apparent structure and their actual governing behavior. Its nearest external comparison fields are governance, software, semantic. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. DOI: 10.1017/CBO9780511807763. https://doi.org/10.1017/CBO9780511807763 **Relationship:** Predecessor institutional account of rule-governed collective action and polycentric governance.

[**E2**] Wil Waluchow and Dimitrios Kyritsis. "Constitutionalism." *Stanford Encyclopedia of Philosophy*, substantive revision 2023. https://plato.stanford.edu/entries/constitutionalism/ **Relationship:** Adjacent legal-philosophical account of limited authority; included to expose differences between political and computational constitutionalism.

[**E3**] Lawrence Lessig. *Code: Version 2.0*. Basic Books, 2006. https://codev2.cc/ **Relationship:** Predecessor argument that software architecture regulates behavior; the paper's contribution may be to make such regulation inspectable and revisable.

[**E4**] OECD. *Recommendation of the Council on Artificial Intelligence*. OECD/LEGAL/0449, adopted 2019 and updated 2024. legalinstruments.oecd.org source **Relationship:** Adjacent policy standard for accountability, traceability, transparency, safety, and human-centered values.

[**E5**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E6**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E7**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1*. NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

Ableman corpus

[**A1**] Adam Ableman. *Ableman Constitutional Governance*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-79353AF163A4B0F1`;
source SHA-256
`FB0D9B2D3D54168FFE70B24308D1B2766D6D663ED24502D475E975939BE97C8B`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Theory.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-D2D97C4042FD281F`;
source SHA-256
`FACC256EDDE5A7D7EA18133CFC9C060CFB8C3811B0221E1969E62347F2BB69CF`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Governance Engineering.* Ableman Corpus current edition; work identity `PAPER.PUB.18749870`;
source SHA-256
`372169F315D192733856B1F4A3F1038BBC73F57CF3ED9ACABAE72B965A69C8D0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Universal Governance Kernel (UGK).* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-1F9BFB2BF8370226`;
source SHA-256
`AA5106EA71E5DF120E141435C280DFC532715CA5DE0C75B8862BB7784ECCFACD`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Legitimacy Without Self-Ratification.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-EF2382F9397335AC`;
source SHA-256
`0186A39C808F8FDDB5C8F42384DF1A26678928434BD50A0EAE8D4947390F68A2`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Sovereign Software.* Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems.* Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Ableman Corpus references [**CC**] Adam Ableman. *Constitutional Compatibility: A Meta-Map for Institutional Semantic Standards.* Exact selected source:
`Constitutional Compatibility.md`,
SHA-256
`FC0C747F474373B385E6F8023B2ECC862CE253A2F2DD7CA45BFE12D87DC90EBB`.
Publication identity remains under review in the corpus ledger.

[**CComp**] Adam Ableman. *Constitutional Computing.* Corpus source family; exact publication and version relation must be resolved through the paper registry before citation finalization.

[**SS**] Adam Ableman. *Sovereign Software: Constitutional Computing and the Reorganization of the Computing Stack.* Corpus source family; exact publication and version relation must be resolved before citation finalization.

[**GMB**] Adam Ableman. *Governed Meaning Base (GMB).* Current corpus terminology under the author-confirmed historical lineage `CCF -> CMB -> GMB`; the terminology ruling grants no publication, canonical, implementation, or conformance standing.

[**GSOL**] Adam Ableman. *Governed Semantic Operations Language* and related MCIR materials. Corpus source family; exact work and version relations remain subject to registry review.

[**SLC**] Adam Ableman. *Semantic Legitimacy Before Context.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

[**RWR**] Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

[**THR**] Adam Ableman. 2026. *The Three Hash Rule.* Zenodo. DOI: `10.5281/zenodo.20788617`.

[**COP**] Adam Ableman. *The Constitutional Observability Platform.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

[**CBT**] Adam Ableman. *Constitutional Binding Theory: Governed Anchor Resolution as a Universal Pattern in Semantic Systems.* Corpus source family; publication does not alter separate canonical standing.

External scholarly and technical sources remain to be researched and verified for a future evidence-qualified revision readiness.

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *The Three Hash Rule.* Zenodo, 2026. doi:10.5281/zenodo.20788617

Adam Ableman *Semantic Debt.* Zenodo, 2026. doi:10.5281/zenodo.21111666

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *Constitutional Computation.* Zenodo, 2026. doi:10.5281/zenodo.20624488

Adam Ableman *Constraint-Bearing Composability.* Zenodo, 2026. doi:10.5281/zenodo.21632061

Adam Ableman *Fractal Governance Architecture.* Zenodo, 2026. doi:10.5281/zenodo.20754189

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848215